Retail Demand Forecasting & Pricing Analytics

Demand, Forecasting, Product, Store & Pricing Analysis

PROJECT BUSINESS REPORT

Disha Chaudhari

Tools: PostgreSQL | SQL | Python | Pandas | CatBoost | Power BI

Dataset: Retail Sales Forecasting Data

Project Type: Descriptive, Diagnostic and Predictive Business Analytics

1. Executive Summary

1.1 Project Overview

This project presents an end-to-end retail analytics solution designed to understand historical demand patterns, evaluate pricing behaviour, forecast future product demand, and translate analytical results into business insights.

The project combines **PostgreSQL, SQL, Python, machine learning, and Power BI** to create a complete analytical workflow:

Raw Retail Data → PostgreSQL → SQL Analysis → Python EDA → Feature Engineering → Demand Forecasting → Model Evaluation → Pricing Analysis → Power BI

The primary objective was to build a practical **machine-learning-based demand forecasting system** while demonstrating the skills expected from a Data Analyst working with large-scale retail data.

The project focuses on:

- Retail demand analysis
- Store and product performance
- Price-demand relationships
- Time-based demand forecasting
- Forecast evaluation and error analysis
- Business-oriented Power BI reporting

The final forecasting model uses **CatBoost Regressor** and achieves:

Metric	Final Result
MAE	**3.37**
RMSE	**8.28**
WAPE	**30.46%**
Forecast Coverage	**100%**

The final CatBoost model was selected because it achieved the best **MAE and WAPE** among the tested model versions, while a rolling-7-day baseline provided the strongest simple benchmark.

2. Business Problem & Objectives

2.1 Business Problem

Retail businesses need to understand how much demand different products and stores are likely to experience in order to support decisions related to inventory, replenishment, sales planning, and pricing.

Historical retail data contains information about:

- Product sales
- Store locations
- Prices
- Dates
- Product categories
- Store formats
- Historical demand

However, raw transactional data alone does not directly answer questions such as:

- Which products and stores generate the most demand?
- How does demand vary over time?
- Which historical demand patterns are useful for forecasting?
- How accurately can future demand be predicted?
- Does recent price information improve demand prediction?
- Where does the forecasting model perform well or poorly?

This project addresses these questions through a combination of SQL analytics, exploratory analysis, machine learning, and visualization.

2.2 Project Objectives

The main objectives were to:

1. Load and organize retail data using **PostgreSQL**.
2. Validate the data structure, grain, duplicates, missing values, and unusual observations.
3. Perform SQL-based business analysis.
4. Explore demand, store, product, time, and pricing patterns using Python.
5. Create time-aware forecasting features.
6. Establish simple forecasting baselines.
7. Develop a **CatBoost demand forecasting model**.
8. Evaluate the model using MAE, RMSE, WAPE, and forecast coverage.
9. Analyze forecasting errors across different demand levels.
10. Build an interactive **Power BI dashboard**.
11. Translate analytical findings into practical business recommendations.

2.3 Key Business Questions

The project focuses on the following questions:

Demand

- How does retail demand change over time?
- Which days of the week have higher demand?
- Which stores contribute the most demand?
- Which products have the highest demand?

Forecasting

- Can historical demand patterns be used to forecast future demand?
- How much improvement does machine learning provide over simple baselines?
- Which features contribute most to forecasting performance?
- Where does the model make the largest errors?

Pricing

- How is demand distributed across different price levels?
- Does recent price information improve demand prediction?
- What can be concluded about price-demand relationships from observational data?

3. Dataset & Data Preparation

3.1 Dataset Overview

The project uses the **Retail Sales Forecasting Data by svizor** dataset obtained from Kaggle.

The dataset consists of multiple CSV files representing different aspects of the retail business, including:

- Sales
- Online sales
- Price history
- Discount history
- Markdowns
- Product catalog
- Store information
- Actual matrix data

The primary sales dataset contains approximately **7.43 million records** and covers the period:

28 August 2022 – 26 September 2024

The sales data contains approximately **28,182 unique products** across **4 stores**.

3.2 Data Sources

The main source files include:

Dataset	Purpose
sales	Main historical retail sales data
online	Online sales information
price_history	Historical product prices
discounts_history	Promotional/discount information
markdowns	Markdown pricing information
catalog	Product attributes
stores	Store attributes
actual_matrix	Additional historical product-store-date information

Not every dataset was directly merged into the forecasting dataset.

This was intentional because the files have different **grains, keys, coverage periods, and business meanings**. Blindly joining them could introduce duplicate rows and distort aggregate measures.

3.3 Data Grain

The main sales data was analysed at the **product-store-date level**.

A key validation was performed on:

date + item_id + store_id

Duplicate excess at this business grain was found to be **zero** in the sales dataset.

Understanding the grain was important because joining datasets with different granularities can result in **row multiplication and incorrect demand or revenue calculations**.

3.4 Data Quality Checks

The data preparation stage included checks for:

- Dataset dimensions
- Data types
- Missing values
- Date ranges
- Duplicate records

- Unique products
- Store coverage
- Negative quantities
- Non-positive prices
- Product catalog matching
- Product-store coverage

The technical source/index column (column0 / Unnamed: 0) was inspected before being excluded from analytical use.

3.5 Important Data Quality Findings

Negative quantities

Negative quantities were present in the raw sales data.

These were treated as potential:

- Returns
- Reversals
- Adjustments

The original records were not destructively removed.

For forecasting, negative quantities were excluded from the target by converting them to missing values.

forecast_df['demand_quantity'] = forecast_df['quantity'].where(

forecast_df['quantity'] >= 0,

np.nan

)

This prevents returns or reversals from being interpreted as normal positive retail demand.

Non-positive prices

Non-positive prices were also observed.

Rather than blindly deleting these records, they were investigated because unusual prices may represent:

- Data issues
- Special transactions
- Adjustments
- Other business-specific cases

Product catalog matching

Approximately **948 item IDs** present in sales did not have a matching catalog record.

These sales records were retained because they contained meaningful sales activity and removing them could unnecessarily reduce the analytical population.

Price history

The price-history dataset contained duplicate date-product-store combinations.

Therefore, it was not blindly joined to the sales data, as doing so could multiply rows and distort demand calculations.

Instead, price-related features were constructed carefully from the analytical sales data.

Auxiliary datasets

Some auxiliary datasets had limitations.

For example, the actual_matrix dataset contained dates extending back to 2019 and had limited exact overlap with the main sales data. Because its analytical role could not be established reliably, it was not forced into the forecasting model.

Similarly, discount data contained suspicious/future dates and was not blindly incorporated.

4. SQL & Exploratory Data Analysis

PostgreSQL was used as the database layer because the project contains millions of retail records and multiple related datasets.

Using PostgreSQL provided a structured environment for:

- Data storage
- Relational analysis
- Data validation
- Aggregation
- Joins
- Business queries
- Analytical data preparation

SQL was used before Python analysis to establish reliable business-level datasets.

The analytical workflow included:

- GROUP BY
- Aggregations
- JOIN
- CASE WHEN
- CTE-based analysis

- Date-based aggregation
- Validation queries
- Product-store analysis

4.2 Overall Demand Analysis

Historical demand was analysed at daily and monthly levels.

Daily analysis examined:

- Total quantity
- Total revenue
- Active products
- Transactions

Monthly analysis examined:

- Total demand
- Total revenue
- Average daily demand
- Average daily revenue
- Active products
- Month-over-month changes

This established the overall demand behaviour before moving into predictive modelling.

4.3 Day-of-Week Demand

Demand was analysed by day of the week.

A clear weekly pattern was observed:

- **Friday** had the highest average demand.
- **Sunday** had the lowest average demand.

This supported the inclusion of calendar features such as:

- Day of week
- Weekend indicator
- Day of month
- Month
- Year

in the forecasting model.

4.4 Store-Level Demand

Demand was analysed across the four stores.

The analysis showed that demand was not evenly distributed across stores.

Stores 1 and 4 contributed a substantially larger share of total demand than the other stores.

This demonstrates why store-level information is important for demand forecasting rather than treating all stores as identical.

4.5 Division Analysis

Demand was also analysed by division.

Division 1 represented the dominant share of demand.

This provides useful business context and also gives the forecasting model additional categorical information about the product's business segment.

4.6 Product Demand

Product-store combinations were found to be relatively sparse and intermittent.

Because not every product-store combination had continuous observations, missing observations could not simply be interpreted as zero demand.

This distinction is important:

Missing observation ≠ zero demand

For forecasting, the modelling population was therefore restricted to product-store combinations with sufficient historical coverage.

A descriptive analysis was performed between price and demand.

The relationship was explored using:

- Positive-price observations
- Positive-demand observations
- Price buckets
- Average demand
- Median demand

The analysis showed that demand behaviour varies across price levels.

However, this project does **not** claim that price changes caused the observed demand changes.

Retail prices can be affected by:

- Promotions
- Product characteristics

- Seasonality
- Store differences
- Demand conditions
- Other business decisions

Therefore, the observed relationship is treated as **predictive/associational rather than causal**.

5. Demand Forecasting

5.1 Forecasting Approach

The forecasting objective was to predict demand at the:

Product × Store × Date

level.

The modelling process followed a chronological structure:

Historical Data

↓

Product-Store Coverage Selection

↓

Feature Engineering

↓

Time-Based Train/Test Split

↓

Baseline Forecasts

↓

CatBoost

↓

Model Evaluation

↓

Final Model Selection

5.2 Forecasting Population

Because the full retail dataset contained many sparse product-store combinations, a coverage filter was applied.

A product-store combination was selected when it had:

- At least **75% coverage**
- At least **180 active days**

This resulted in approximately:

5,553 product-store pairs

Even after filtering, some gaps remained, which is why missing observations were not automatically treated as zero demand.

5.3 Demand Target

The forecasting target was:

demand_quantity

Negative quantities were excluded from the modelling target because they were considered more representative of returns/reversals/adjustments than normal demand.

The final training dataset contained approximately:

2.45 million target-valid observations

5.4 Feature Engineering

The forecasting model used **20 features** covering historical demand, pricing, calendar information, and categorical context.

Demand history

- lag_1
- lag_7
- lag_28
- rolling_7
- rolling_28
- rolling_7_count
- rolling_28_count
- days_since_previous

Price

- price_lag_1
- price_change

Calendar

- day_of_week
- day_of_month

- month
- year
- is_weekend

Business context

- item_id
- store_id
- store_format
- city
- division

5.5 Calendar-Aware Lag Features

The lag features were constructed using exact calendar dates rather than simply taking the previous observed row.

For example:

lag_1 = demand from the previous calendar day.

This is important because retail data contains gaps.

A positional shift could incorrectly treat the previous available observation as "yesterday" even when several days were missing.

The same approach was applied to:

- 1-day lag
- 7-day lag
- 28-day lag

5.6 Rolling Features

Seven-day and 28-day rolling demand features were created.

The rolling calculations used:

closed='left'

This ensured that the current day's target was not included when calculating the historical rolling average.

This prevents **target leakage**.

Rolling counts were also created to indicate how much historical evidence was available within each rolling window.

5.7 Time-Based Train/Test Split

A chronological split was used:

Training:

Before 1 July 2024

Testing:

1 July 2024 onward

The split was intentionally time-based rather than random.

Random train/test splitting is inappropriate for this forecasting problem because future observations could indirectly influence training data.

The model therefore learned from the past and was evaluated on a later period.

5.8 Baseline Models

Before using machine learning, three simple baselines were evaluated.

Model	MAE	RMSE	WAPE
Lag 1	4.56	11.10	39.11%
Lag 7	4.55	9.46	38.91%
Rolling 7	**3.72**	9.53	33.52%

The **7-day rolling average** was the strongest simple baseline.

This provided an important benchmark for determining whether the machine-learning model actually added value.

5.9 CatBoost Model

CatBoost Regressor was selected because the problem is structured tabular data containing:

- Product identifiers
- Store identifiers
- Categorical attributes
- Time features
- Lagged demand
- Rolling statistics
- Price features

The final model configuration was:

Parameter	Value
Model	CatBoost Regressor
Iterations	500

Depth	8
Learning Rate	0.05
Loss Function	RMSE
Random Seed	42

Categorical features included:

- item_id
- store_id
- store_format
- city
- division

5.10 Model Iterations

The forecasting model was developed incrementally.

V1 — Initial CatBoost

Used demand, calendar, and business-context features.

V2 — Store ID Added

Adding store_id slightly improved the model, particularly RMSE.

V3 — Previous Price Added

price_lag_1 was introduced to capture recent pricing information.

V4 — Price Change Added

price_change was added alongside price_lag_1.

The final V4 model achieved the best **MAE and WAPE**.

5.11 Final Model Selection

The model comparison was:

Model	MAE	RMSE	WAPE
V2	3.42	**8.11**	30.88%
V3	~3.41	~8.31	~30.77%
V4	**3.37**	8.28	**30.46%**

V2 achieved the lowest RMSE, but V4 was selected because it achieved the best:

- MAE
- WAPE

For this project, reducing average absolute forecast error and overall proportional error was considered more useful than selecting a model solely because of the lowest RMSE.

6. Forecast Performance

6.1 Evaluation Metrics

Three primary error metrics were used.

MAE

Mean Absolute Error measures the average absolute difference between actual and predicted demand.

The final:

MAE = 3.37

means the model's predictions were off by approximately 3.37 demand units on average.

RMSE

Root Mean Squared Error gives greater weight to larger errors.

The final:

RMSE = 8.28

The difference between MAE and RMSE indicates the presence of some larger forecasting errors.

WAPE

Weighted Absolute Percentage Error was calculated as:

Total Absolute Forecast Error ÷ Total Absolute Actual Demand × 100

Final:

WAPE = 30.46%

WAPE should **not** be interpreted as "69.54% forecast accuracy."

It is an aggregate error measure.

6.2 Final Model Performance

Metric	Final CatBoost
MAE	**3.37**
RMSE	**8.28**
WAPE	**30.46%**
Coverage	**100%**

Compared with the rolling-7 baseline:

WAPE improved from **33.52% → 30.46%**

This indicates that CatBoost provided a meaningful improvement over the strongest simple baseline.

6.3 Forecast Error by Demand Band

The test data was divided into four demand bands.

Demand Band	MAE	RMSE	WAPE
Low	2.06	3.12	146.41%
Medium	1.70	2.86	44.62%
High	2.63	3.92	34.78%
Very High	7.70	15.58	**22.89%**

Key observation

Low-demand observations produced a very high WAPE.

This is largely because percentage-based errors become unstable when actual demand values are small.

For example, an error of only a few units can represent a very large percentage of a low-demand observation.

At the other end, the **Very High** demand band had the largest absolute errors but the lowest WAPE.

This illustrates why forecasting performance should be evaluated using multiple metrics rather than one metric alone.

6.4 Feature Importance

The most important V4 features were approximately:

Feature	Importance
lag_1	38.93
price_lag_1	30.03
rolling_7	6.37
day_of_week	5.38
lag_7	5.33
rolling_28	3.03
lag_28	2.08
month	2.02

day_of_month	1.87
item_id	1.32
price_change	0.74

The model was strongly influenced by **recent demand history**, particularly lag_1.

Recent price information was also highly predictive.

However:

Feature importance indicates predictive usefulness, not causality.

Therefore, the importance of price_lag_1 does not prove that changing price will cause a corresponding change in demand.

7. Power BI Dashboard

The final Power BI dashboard contains three pages designed around business questions rather than simply displaying charts.

7.1 Page 1 — Executive Overview

The executive page provides a high-level view of demand and forecasting performance.

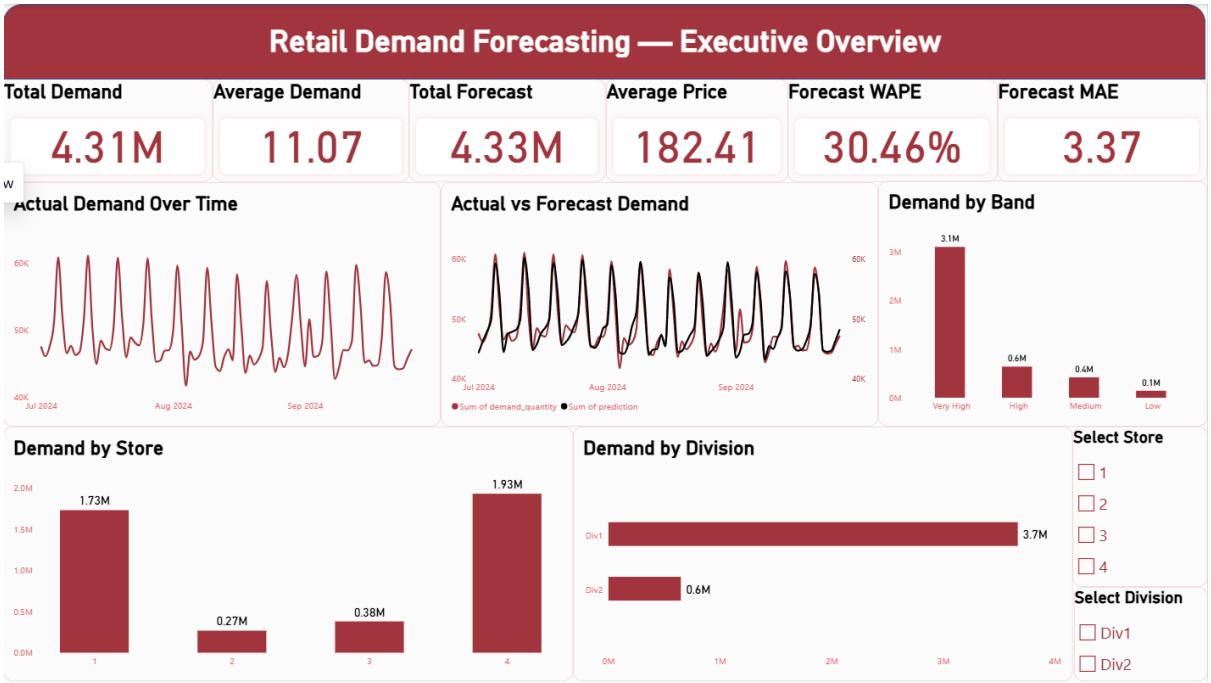
KPIs

KPI	Value
Total Demand	**4.31M**
Average Demand	**11.07**
Total Forecast	**4.33M**
Average Price	**182.41**
Forecast WAPE	**30.46%**
Forecast MAE	**3.37**

Visuals

- Actual vs Forecast Demand over Time
- Demand by Demand Band
- Demand by Store
- Demand by Division
- Store slicer
- Division slicer

This page is intended for quickly understanding the overall demand position and model performance.



7.2 Page 2 — Demand & Pricing Analysis

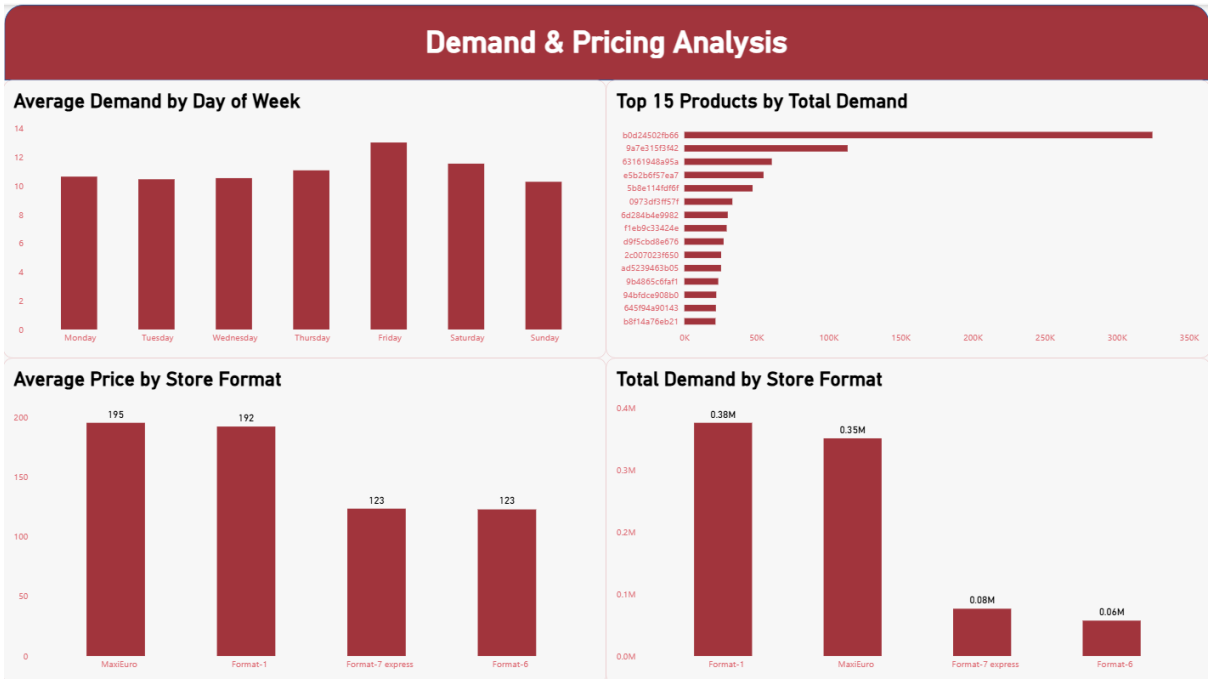
This page focuses on demand patterns and pricing context.

Visuals

- Average Demand by Day of Week
- Top 15 Products by Total Demand
- Average Price by Store Format
- Total Demand by Store Format

A price-range visualization was avoided because the price distribution was highly skewed.

Average price by store format provides a more interpretable comparison.



7.3 Page 3 — Forecast Performance

This page focuses specifically on model behaviour.

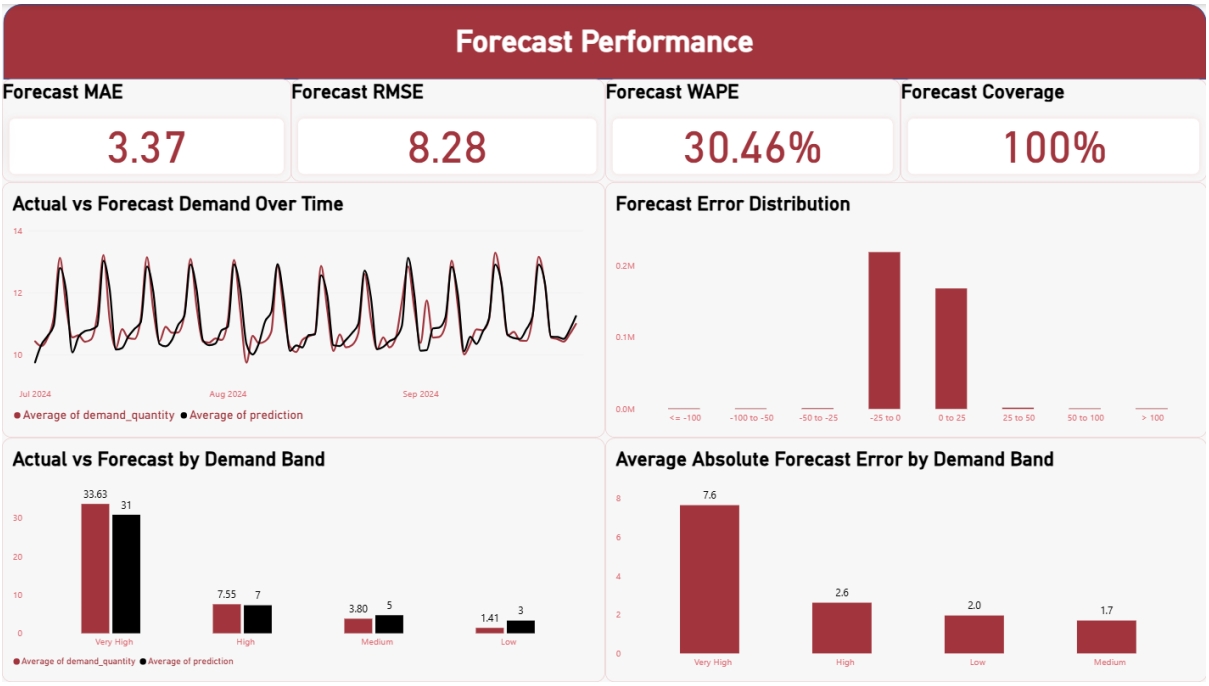
KPIs

- MAE — **3.37**
- RMSE — **8.28**
- WAPE — **30.46%**
- Coverage — **100%**

Visuals

- Actual vs Forecast Demand over Time
- Forecast Error Distribution
- Actual vs Forecast by Demand Band
- Average Absolute Forecast Error by Demand Band

This page allows users to distinguish between overall model performance and performance across different demand levels.



8. Business Insights & Recommendations

8.1 Demand Planning

Recent demand history was the strongest predictive signal in the forecasting model.

Therefore, short-term demand planning should place significant emphasis on recent sales behaviour rather than relying only on long-term averages.

Recommendation

Use recent demand history as a primary input for short-term replenishment and planning decisions.

8.2 Store-Level Planning

Demand was unevenly distributed across stores, with Stores 1 and 4 contributing substantially more demand.

Recommendation

Demand planning should be **store-aware** rather than applying a single demand assumption across all stores.

8.3 Weekly Demand Patterns

Friday showed the highest average demand while Sunday showed the lowest.

Recommendation

Inventory and staffing decisions can incorporate weekly demand patterns to better align resources with expected activity.

8.4 Forecasting

The CatBoost model improved upon the rolling-7 baseline:

WAPE: 33.52% → 30.46%

Recommendation

Machine-learning forecasting can complement simple statistical baselines when historical demand, product, store, calendar, and pricing information are available.

However, the baseline should always be retained as a benchmark.

8.5 Forecast Error Monitoring

The model behaved differently across demand levels.

Low-demand observations had very high WAPE, while very-high-demand observations had larger absolute errors.

Recommendation

Forecast monitoring should segment errors by demand level rather than relying only on one overall metric.

8.6 Pricing

Recent price information was one of the strongest predictors in the model.

However, this does **not** establish a causal price elasticity relationship.

Recommendation

Price information can be used as a forecasting feature, but pricing decisions should not be based on this model alone.

A dedicated causal pricing analysis would require additional controls for promotions, product characteristics, seasonality, store effects, and other confounding factors.

9. Limitations & Future Improvements

9.1 Data Limitations

The dataset contains several challenges:

- Sparse product-store observations
- Negative quantities
- Non-positive prices
- Incomplete product catalog matching
- Different coverage across auxiliary datasets
- Potential data-quality issues in promotional datasets

These limitations affect how confidently the results can be generalized.

9.2 Forecasting Limitations

The forecasting model was evaluated using a chronological holdout period.

While this is more appropriate than random splitting, a production forecasting system would benefit from repeated **walk-forward validation** across multiple historical periods.

The model also focuses on a selected population of sufficiently observed product-store combinations rather than every possible product-store pair.

9.3 Pricing Limitations

The price-demand relationship is based on observational retail data.

Prices are not randomly assigned, and pricing decisions can themselves be influenced by expected demand, promotions, inventory, and product characteristics.

Therefore, this project does **not** claim:

- Causal price elasticity
- Optimal pricing
- Revenue-maximizing prices
- A causal impact of price changes

9.4 Future Improvements

Potential future improvements include:

1. Walk-forward model validation.
2. More robust price elasticity modelling.
3. Promotion-aware forecasting.
4. Stockout-aware demand modelling.
5. Price scenario simulation.
6. Additional external demand drivers.
7. More advanced forecasting models for comparison.
8. Automated model monitoring and retraining.

These are extensions rather than completed components of the current project.

10. Conclusion

This project demonstrates an end-to-end retail analytics workflow combining **PostgreSQL, SQL, Python, machine learning, and Power BI**.

The analysis began with data validation and SQL-based exploration before moving into exploratory analysis, time-aware feature engineering, forecasting, and business visualization.

Three simple forecasting baselines were evaluated before developing progressively improved CatBoost models. The final V4 model achieved:

MAE: 3.37
RMSE: 8.28
WAPE: 30.46%

and achieved **100% forecast coverage** on the evaluation dataset.

The analysis also showed that recent demand history was the strongest predictive signal, while recent price information provided substantial additional predictive value.

At the same time, the project deliberately avoids overstating the results. Price-related relationships are treated as **predictive associations rather than causal effects**, and the forecasting model is not presented as an "optimal pricing engine."

Overall, the project demonstrates practical skills across:

SQL & PostgreSQL → Data Analysis → Feature Engineering → Machine Learning → Forecast Evaluation → Power BI → Business Recommendations

The result is a practical retail analytics project that connects technical analysis with business decision-making rather than treating machine learning as an isolated modelling exercise.